

Manual de Cypress Epmsa

Instalación en Ubuntu

Requisitos previos

Para la instalación de Cypress existen requisitos previos que el usuario debe cumplir como:

- **Node.js (versión 16 o superior recomendada)**
- **npm (administrador de paquetes de Node)**
- **git** (Para descargar el proyecto de odoo-cypress-tests)

Instalar con:

```
sudo apt update
sudo apt install nodejs npm git -y
node -v
npm -v
```

Instalación de Cypress

1. `git clone git@git.katarisoft.com:epmsa/odoo-cypress-tests.git`
2. `cd odoo-cypress-tests`
3. `npm install`
4. Crear archivo `.env`
- 5.

1. Crear carpeta para el proyecto:

```
mkdir proyecto-cypress && cd proyecto-cypress
npm init -y
```

2. Instalar Cypress:

```
npm install cypress --save-dev
```

3. Abrir Cypress por primera vez:

```
npx cypress open
```

Esto abrirá la interfaz de Cypress y generará la estructura de carpetas.

Configuración inicial

En `package.json` puedes agregar un script para ejecutar Cypress:

```
"scripts": {
  "test:e2e": "cypress open",
  "test:run": "cypress run"
}
```

Librerías y herramientas complementarias

Cypress incluye capturas y videos de forma nativa:

- **Capturas de pantalla:** se guardan en `cypress/screenshots/`.
- **Videos de ejecución:** se guardan en `cypress/videos/`.

Reportes

Para generar reportes se recomienda Mochawesome.

Instalación:

```
npm install --save-dev mochawesome mochawesome-merge mochawesome-report-generator
```

Configurar en `cypress.config.js`:

```
const { defineConfig } = require("cypress");

module.exports = defineConfig({
  reporter: "mochawesome",
  reporterOptions: {
    reportDir: "cypress/reports",
    overwrite: false,
    html: true,
    json: true
  }
});
```

Estructura de un proyecto Cypress

Cuando instalas Cypress, se crea la siguiente estructura:

```
cypress/
├── e2e/           # Aquí van todos los tests
├── fixtures/      # Archivos JSON con datos de prueba
├── support/       # Comandos y configuraciones globales
├── downloads/     # Archivos descargados
├── screenshots/   # Capturas automáticas
├── videos/        # Grabaciones de las pruebas
└── cypress.config.js # Configuración principal de Cypress
```

Explicación de carpetas

- **e2e/:** contiene los casos de prueba (ej: `login.cy.js`).
- **fixtures/:** almacena datos de prueba (ej: usuarios, contraseñas).
- **support/:** incluye código compartido y comandos personalizados.
- **screenshots/:** Cypress guarda capturas aquí cuando un test falla.
- **videos/:** Cypress graba videos automáticamente de las ejecuciones.

Capítulo 5: Comandos básicos de Cypress (20+)

1. `cy.visit(url)`

Abre una página web.

```
cy.visit('https://example.com')
```

2. **cy.get(selector, options)**

Selecciona un elemento del DOM.

```
cy.get('button#login', { timeout: 10000 })
```

- 1er argumento: selector CSS.
- 2do argumento: opciones (ejemplo: tiempo de espera).

3. **cy.contains(text)**

Selecciona un elemento por el texto que contiene.

```
cy.contains('Iniciar sesión')
```

4. **cy.click()**

Hace click en un elemento.

```
cy.get('button#login').click()
```

5. **cy.type(text)**

Escribe en un campo de texto.

```
cy.get('input#username').type('usuario123')
```

6. **cy.clear()**

Limpia el campo de texto.

```
cy.get('input#username').clear()
```

7. **cy.select(value)**

Selecciona un valor en un <select>.

```
cy.get('select#pais').select('Ecuador')
```

8. **cy.check()**

Marca un checkbox o radio button.

```
cy.get('input[type="checkbox"]').check()
```

9. **cy.uncheck()**

Desmarca un checkbox.

```
cy.get('input[type="checkbox"]').uncheck()
```

10. **cy.should(assertion, value)**

Hace una aserción sobre el elemento.

```
cy.get('h1').should('contain', 'Bienvenido')
```

11. cy.url()

Obtiene la URL actual.

```
cy.url().should('include', '/dashboard')
```

12. cy.title()

Obtiene el título de la página.

```
cy.title().should('eq', 'Mi aplicación')
```

13. cy.wait(time)

Espera un tiempo en milisegundos.

```
cy.wait(2000)
```

14. cy.intercept(method, url)

Intercepta una petición de red.

```
cy.intercept('GET', '/api/usuarios').as('usuarios')  
cy.wait('@usuarios')
```

15. cy.fixture(filename)

Carga datos de prueba desde fixtures.

```
cy.fixture('usuario.json').then((user) => {  
  cy.get('#username').type(user.name)  
})
```

16. cy.screenshot(name)

Toma una captura de pantalla.

```
cy.screenshot('login-error')
```

17. cy.wrap(obj)

Convierte un objeto en cadena Cypress.

```
cy.wrap([1, 2, 3]).should('include', 2)
```

18. cy.task(name, arg)

Ejecuta una tarea en Node.js.

```
cy.task('log', 'Prueba Cypress')
```

19. cy.exec(command)

Ejecuta un comando del sistema.

```
cy.exec('echo hola mundo')
```

20. cy.log(message)

Escribe un mensaje en el log de Cypress.

```
cy.log('Inicio del test de login')
```

21. cy.viewport(width, height)

Cambia el tamaño de la pantalla.

```
cy.viewport(1280, 720)
```

Capítulo 6: Hooks en Cypress

before()

Corre una vez antes de todos los tests.

```
before(() => {  
  cy.log('Preparando datos de prueba...')  
})
```

beforeEach()

Corre antes de cada test individual.

```
beforeEach(() => {  
  cy.visit('/login')  
})
```

after()

Corre una vez al finalizar todos los tests.

```
after(() => {  
  cy.log('Finalizando pruebas')  
})
```

afterEach()

Corre después de cada test individual.

```
afterEach(() => {  
  cy.log('Terminó un test')  
})
```

Capítulo 7: Uso de cy.task

cy.task sirve para ejecutar funciones de Node.js desde Cypress.

Esto es útil cuando necesitas interactuar con archivos, bases de datos o hacer logs personalizados.

Ejemplo de configuración en cypress.config.js

```
setupNodeEvents(on, config) {  
  on('task', {  
    log(message) {  
      console.log(message)  
      return null  
    }  
  })  
}
```

Ejemplo de uso en un test

```
cy.task('log', 'Esto viene desde Cypress al servidor Node.js')
```

Capítulo 8: Buenas prácticas en Cypress

- Usar `beforeEach()` para inicializar estado.
- Evitar `cy.wait(5000)`: mejor usar `cy.intercept()`. (Solo se debe usar `cy.wait` cuando exista un problema de carga en odoo)
- Separar tests por módulos (ej: login, dashboard, perfil).
- Centralizar datos en fixtures.
- Usar comandos custom para no repetir código:

```
Cypress.Commands.add('login', (usuario, pass) => {  
  cy.get('#username').type(usuario)  
  cy.get('#password').type(pass)  
  cy.get('button#login').click()  
})
```

Ejemplo práctico completo

Archivo: cypress/e2e/login.cy.js

```
describe('Prueba de login', () => {  
  beforeEach(() => {  
    cy.visit('/login')  
  })  
  
  it('Debería iniciar sesión correctamente', () => {  
    cy.get('#username').type('admin')  
    cy.get('#password').type('1234')  
    cy.get('#login').click()  
    cy.url().should('include', '/dashboard')  
  })  
  
  it('Debería mostrar error con credenciales inválidas', () => {  
    cy.get('#username').type('invalido')  
    cy.get('#password').type('error')  
    cy.get('#login').click()  
    cy.contains('Usuario o contraseña incorrecta')  
  })  
})
```